



Universitat
Oberta
de Catalunya

(HHTMLCSS1) Herramientas HTML y CSS I
PEC 3 - Rendimiento web

Marc Turu Roca

Profesor: Mariona Roca Ros

Fecha de entrega: 12/6/2026

Universitat Oberta de Catalunya (UOC)

Tabla de contenidos

0. Introducción	2
1. Tabla y análisis del tiempo de carga	2
2. Primeros cambios	6
3. Informe de mejoras	8
4. Preguntas finales	11
4.1 Lazy loading	11
4.2 Carga asíncrona de scripts	12
4.3 Carga asíncrona de css	13
5. Anexos	15

0. Introducción

Esta práctica ha consistido en la mejora del sitio web desarrollado en las prácticas 1 y 2. Para ello, se han elaborado análisis de rendimiento de la página web y se han aplicado mejoras de rendimiento (sugeridas en parte por [Google PageSpeed Insights](#)). Todo esto asegurando la adecuación de estándares y calidad de código en general.

Es conveniente recalcar que el estado de la web antes de empezar a realizar esta práctica era, por un lado, con las técnicas de **lazy loading** y **carga asíncrona de los scripts** ya aplicadas, por lo que se decidió deshacerlas para poder tomar las medidas antes y después de utilizarlas, y por otro lado, con una puntuación inicial de **97 rendimiento** en móvil y **100** en ordenador, **96 accesibilidad**, **100 prácticas recomendadas** y **100 SEO**, por lo que se aplicaron las mejoras propuestas por el test a partir de estos valores.

El repositorio de **GitHub** es: <https://github.com/marcturu/saboreamexico>

El link para acceder a la **web** es: <https://saboreamexico.netlify.app/>

1. Tabla y análisis del tiempo de carga

Para realizar las comparaciones en cada fase, se decidió utilizar las siguientes métricas de *Network* de *DevTools* de *Firefox Developer*¹ desde la propia URL pública de la aplicación web. De esta manera, se podían obtener los datos más fiables posibles respecto al entorno real del usuario final y estos utilizaban el mismo entorno que [Google PageSpeed Insights](#):

- **Regular 3G**. Porque simula condiciones de red móviles lentas y permite evaluar el rendimiento en escenarios de conexión limitada.
- **“Disable caché”**. Porque fuerza la recarga completa de los recursos desde el servidor sin usar caché del navegador.
- **Tiempo** de carga de la página completa (específicamente el *load*, ya que representa la carga real de la página sin depender de *tracking* externo y recursos no críticos, y es cuando deja de transferirse contenido), **peso total** de la página, **peso transferido** que viajó por la red y **cantidad de recursos** cargados.

Para ser más preciso, cada medición se realizó 7 veces, se desestimó el valor más alto y más bajo y se tomó la media aritmética de los 5 restantes.

En la siguiente tabla, se compara el estado inicial (1) con el estado optimizado (2) tras aplicar **lazy loading** y **carga asíncrona de scripts**, con el estado final (3) tras aplicar las recomendaciones de mejora de los tests de **Google PageSpeed Insights**

¹ Estos valores se tomaron sin hacer *scroll* para poder tener más robustez y fiabilidad en los resultados.

Título	URL	Tiempo de carga (s)			Peso total (kB)			Peso transferido (kB)			Recursos		
		1	2	3	1	2	3	1	2	3	1	2	3
Inicio	/	5,32	3,32	3,30	826,46	643,43	644,60	705,30	522,44	522,76	15	12	12
Recetas	/categoria	5,37	4,45	3,32	894,93	736,89	637,94	774,39	616,54	516,75	16	13	13
Tacos	/det1	18,93	3,35	4,36	5150,72	711,00	822,37	2017,28	590,45	700,81	41	13	14
Nachos	/det2	17,13	3,22	4,31	5017,60	632,81	744,18	1873,92	512,45	622,79	41	13	14
Fuentes	/links	2,41	2,31	2,30	560,58	559,25	560,40	436,32	435,24	435,51	12	11	11

Páginas de contenido general (/ , /categoria, /links)

En las páginas de inicio y categoría se observa una mejor muy significativa entre la fase 1 y la fase 2. La página pasó de 5,32s a 3,32s (-37,6%), reduciendo el peso transferido de 705,30 kB a 522,44 kB (-25,9%) y los recursos de 15 a 12. La página de categoría mejoró de 5,37s a 4,45s en la fase 2, y continuó optimizándose hasta 3,32 en la fase 3 (-38,2% respecto al inicio), con una reducción de 16 a 13 y del peso transferido de 774,39 kB a 516,75 kB (-33,3%).

La página /links, al no contener imágenes ni animaciones AOS relevantes, muestra valores estables en la tres fases (2,41s -> 2,31s -> 2,30s), lo que confirma que las optimizaciones aplicadas afectaron principalmente a recursos multimedia y scripts de terceros.

Páginas de detalle (/det1 y /det2)

Estas páginas presentan los cambios más llamativos. En la fase 1, /det1 registró 18,93s de tiempo de carga y un peso transferido de 2017,28 kB. Este valor extremo se explicaba por la presencia del iframe de YouTube sin lazy loading. En una conexión 3G simulada, el navegador descargaba todos los recursos de YouTube (scripts, CSS, fuentes) antes de disparar el video **load**, lo que bloqueaba la medición. det2 presentó un comportamiento idéntico con 17,13s y 1873,92 kB.

Tras aplicar aplicar el lazy loading al iframe del vídeo de YouTube, los tiempos se redujeron drásticamente: /det1 pasó a 3,35s (-82,3%) y /det2 a 3,22 (-81,2%), con el peso transferido cayendo a 590,45 kB y 512, 45 kB, respectivamente. El número de recursos bajó de 41 a 13 en ambos casos, lo que reflejaba directamente la eliminación de todas las peticiones de red generadas por el iframe en la carga inicial.

Variación entre fase 2 y fase 3 en /det1 y /det2

Entre la fase 2 y la fase 3 se observó un ligero incremento tanto en el tiempo de carga (de 3,35s a 4,36s en /det1 y de 3,22s a 4,31s en /det2) como en el peso transferido y e número de recursos (de 13 a 14). Este incremento era atribuible a la incorporación del preload de fuentes críticas implementado como mejora recomendada por PageSpeed Insights, concretamente la adición de fa-brands-400.woff2 al conjunto de recursos precargados. Esta fuente, utilizada por Font Awesome para los iconos utilizados, no se cargaban automáticamente porque el navegador la descubre tarde en la cadena de dependencias CSS -> @font-face. Al implementar el preload, la fuente se descargaba de forma anticipada y prioritaria, lo que la hacía visible en la pestaña de red desde el inicio de la carga. Aunque esto incrementaba ligeramente el peso transferido inicial, el objetivo del preload era precisamente este: adelantar la descarga de recursos críticos para reducir la latencia percibida y mejorar métricas como FCP y LCP, a costa de un mayor peso transferido inicial.

En términos de rendimiento real percibido por el usuario, esta fase 3 representaba una mejora cualitativa respecto a la fase 2, ya que el texto y los iconos se renderizar antes al estar la fuente disponible desde el inicio, eliminando el FOIT (Flash of Invisible Text) causado por la carga tardía de Font Awesome.

Conclusión

Las optimizaciones aplicadas supusieron una mejora global muy significativa. La reducción mas relevantes se produjo entre la fase 1 y la fase 2 gracias a la combinación de 2 técnicas: lazy loading en imágenes y carga asíncrona de scripts mediante `type="module"` (equivalente a defer por especificación). La fase 3 añadió optimizaciones más finar orientadas a las métricas Core Web Vitals, con un impacto menor en los tiempo absolutos pero con mejoras en la experiencia de carga percibida.

2. Primeros cambios

Después de simular una fase inicial sin optimizaciones quitando lazy loading y carga asíncrona de scripts mediante la creación de un único archivo JavaScript en vez de la utilización de módulos como se realizó en la PEC anterior, para esta segunda fase sí que se aplicaron **lazy loading** y **carga asíncrona de scripts**.

Los lazy loading se aplicaron en aquellos recursos no críticos para el render inicial de la página, principalmente en imágenes y vídeos situados por debajo del fold (contenido no visible inicialmente en pantalla). En estos casos, se utilizó la carga diferida mediante `loading="lazy"`, en las etiquetas `<img>` en las 2 imágenes de la página /, en las 2 últimas imágenes de la página /**categoria**, y en las 2 últimas imágenes tanto de la página /**det1** como /**det2**. Y en las etiquetas `<iframe>` en los dos vídeos incrustados de YouTube en las páginas /**det1** como /**det2**.

El objetivo fue retrasar su descarga hasta que el usuario se desplazara por la página, reduciendo así el tiempo de carga inicial y el consumo de ancho de banda. A continuación se presenta un ejemplo de su aplicación para una imagen de la portada:

```

```

Por otro lado, se quitó el CDN AOS utilizado anteriormente para la carga de dicha dependencia y se aplicó la carga asíncrona de scripts para evitar que se bloqueara el parsing del HTML y mejorara el tiempo de interactividad.

Para ello, se usó el tipo asíncrono/diferido `type="module"` para cargar el archivo **app.js**:

```
<script type="module" src="./assets/scripts/app.js"></script>
```

que manejaba los 2 módulos a cargar:

```
import { initHeader } from './modules/header.js'  
import { initAOS } from './modules/aos.js'  
  
initHeader();  
initAOS();
```

En resumen, se pasó de utilizar un único archivo monolítico **app-legacy.js** que contenía toda la lógica de la aplicación, a una arquitectura modular basada en **app.js** y módulos ES que actuaba como punto de entrada de la aplicación e importaba los módulos necesarios. Es imperativo recalcar que los scripts de tipo módulo se descargan de forma no bloqueante y se ejecutan una vez finalizado el parseado del documento y solo cuando el DOM está completo, **ofreciendo un comportamiento equivalente a `defer` de forma nativa**. Además, se dividía el código en módulos independientes y cargaba únicamente las dependencias necesarias.

En conjunto, estas optimizaciones permitían reducir el impacto de recursos secundarios en el rendimiento percibido, priorizando el contenido crítico y mejorando las métricas de carga inicial.

3. Informe de mejoras

Las pruebas se ejecutaron para cada una de las cinco páginas del proyecto en versión móvil y escritorio, antes y después de aplicar las mejoras. Cada medición se realizó 7 veces, descartando el valor más alto y el más bajo, tomando la media aritmética de los 5 restantes para minimizar el ruido de red.

URL	Resultado móvil antes	Resultado móvil después	Resultado escritorio antes	Resultado escritorio después
/	97	98	100	100
/categoria	93	97	100	100
/det1	89	91	87	100
/det2	90	95	86	99
/links	99	99	100	100

- Para la **solicitud de bloqueo de la renderización** (presente en **todas las páginas**) se sugirió *diferir o insertar inline los recursos CSS que bloqueaban el camino crítico de renderización*, lo cual era difícil de mejorar porque, al tener bundles de CSS generados automáticamente con Parcel (repo.*.css), habría que implementar critical CSS (inline CSS above-the-fold en el `<head>` y cargar el resto asyncs), lo cual es complejo con Parcel sin plugins adicionales. De todas maneras, se añadió un `rel="preload"` en el `<head>` de modo que el navegador iniciaba la descarga del CSS en paralelo con el resto de recursos desde el primero momento, reduciendo la latencia:

```
<link rel="preload" href="./assets/styles/main.scss" as="style" />
<link rel="stylesheet" type="text/css" href="./assets/styles/main.scss" />
```

- Para la **visualización de la fuente** (presente en **todas las páginas**) ya tenía aplicada la sugerencia que se proponía: `font-display: swap` en las fuentes propias. El problema residual venía de Font Awesome (librería externa), cuyo `@font-face` no es modificable sin self-hosting.
- La **recomposición forzada** (presente en **todas las páginas menos /categoria**) provenía del bundle de AOS, la librería externa que internamente consulta propiedades geométricas del DOM. No era modificable sin sustituir la librería, por lo

que se documentó como limitación conocida de AOS con impacto de 43ms, considerado no crítico dada la puntuación de rendimiento obtenida.

- En el **árbol de dependencias de la red** (presente en **todas las páginas**) se acortó la latencia máxima moviendo las descargas al inicio con el `preload` en los `.html` (como pedía la sugerencia):

```
<link rel="preload" href="./assets/fonts/unbounded-v12-latin-700.woff2"
as="font" type="font/woff2" crossorigin>
```

- Para la **mejora de publicación de imágenes** (presente en **todas las páginas**), se solucionó entendiendo que algunos de los ficheros que se servían eran más grandes que las dimensiones que se mostraban. Por ejemplo en las imágenes de las tarjetas, 1200x726 de imagen por 637x529 de dimensiones, por lo que se eliminó del código:

```
./assets/images/recipes/nachos.jpg?width=1200&as=webp 1200w
```

y se modificó:

```
(max-width: 768px) 100vw,
(max-width: 1200px) 50vw,
600px
```

Igual que cambiar el `sizes="1000px"` por `sizes="100vw"` en los *heros* de los `/det1` `/det2`.

- Para la **reducción del número de ficheros CSS no utilizados** (presente en **todas las páginas**), el problema era que el `iframe` de YouTube cargaba `~107 KiB CSS`. La solución fue implementar un **facade de YouTube** en el que se mostraba una imagen estática con botón de play, y se cargaba el `iframe` real solo cuando el usuario hacía click. Todo esto aplicando estilos CSS y funcionalidades JS propias.
- Para la **reducción del número de ficheros JavaScripts no utilizados** (presente en `/det1` y `/det2`), el problema también era que el `iframe` de YouTube cargaba `~783 KiB JS`. La solución fue la misma que para CSS.
- Para la **utilización de tiempo de vida eficiente de la caché** (presente en `/det1` y `/det2`) el header de caché lo controla i.ytimg.com, que son servicios de Google, por lo que no se pudo modificar y representó una limitación de recurso externo. Un problema curioso también es que i.ytimg.com no soporta `srcset` porque es un dominio externo, por lo que no se le puede pedir versiones distintas. Aún así, con `sizes` al menos se pudo documentar la intención (aunque el navegador siguiese cargando la misma imagen).
- La **latencia de la solicitud del documento** (presente en `/det2`) es probable que fuera Netlify respondiendo lento en esa URL específica, algo que no se puede controlar desde el código ya que depende de la CDN de Netlify y el plan gratuito.

A continuación se ven los valores de las **Core Web Vitals** finales²:

URL	FCP (s)		LCP (s)		CLS	
	Móvil	Ordenador	Móvil	Ordenador	Móvil	Ordenador
/	1,2	0,3	2,2	0,4	0	0
/categoria	1,1	0,4	2,5	0,5	0	0
/det1	1,7	0,3	2,8	0,7	0	0,004
/det2	1,1	0,3	2,8	0,5	0	0,01
/links	1,1	0,4	1,9	0,4	0	0,006

² Se substituyó INP (Interaction to Next Paint) por FID (First Input Delay) debido a que Lighthouse no lo mostraba explícitamente.

4. Preguntas finales

4.1 Lazy loading

¿Qué cambios detectas en las herramientas para desarrolladores al aplicar *lazy loading* a las imágenes de tu web? ¿Cómo crees que afecta al rendimiento de tu página?

Al aplicar **loading="lazy"** a las imágenes, los cambios observables en la pestaña Network de las herramientas para desarrolladores son inmediatos y significativos.

En la fase sin lazy loading, todas las imágenes de la página aparecen en el waterfall de red desde el primer momento de la carga, independientemente de si están visibles en el viewport inicial o se encuentran a varios scrolls de distancia. En la fase con lazy loading, las imágenes fuera del viewport simplemente no aparecen en la pestaña Network en la carga inicial. Solo se registran nuevas peticiones a medida que el usuario hace scroll y los elementos se aproximan al viewport, momento en el que el navegador inicia su descarga de forma automática gracias al **IntersectionObserver** que implementa el propio navegador internamente.

Esto se traduce en tres cambios medibles directos: reducción del número de recursos en la carga inicial (en este proyecto, de 41 a 13 en **/det1** y **/det2**), reducción del peso transferido inicial y reducción del tiempo hasta el evento **load**, ya que este evento no se dispara hasta que todos los recursos solicitados han terminado de descargarse.

En cuanto al rendimiento, el impacto es positivo en varias dimensiones. Al reducir el número de peticiones simultáneas en la carga inicial, el navegador puede priorizar los recursos críticos: CSS, fuentes, scripts y las imágenes above-the-fold, sin competir por el ancho de banda con imágenes que el usuario aún no ha visto. Esto mejora métricas como el **FCP** (*First Contentful Paint*) y el **LCP** (*Largest Contentful Paint*), que miden la velocidad a la que el contenido visible se renderiza. El caso más extremo en este proyecto fue el iframe de YouTube sin **loading="lazy"**, que en conexión 3G simulada elevaba el tiempo de carga de **/det1** a 18,93s; tras aplicar la facade con lazy loading, se redujo a 3,35s.

Es importante señalar que el lazy loading no debe aplicarse indiscriminadamente. Las imágenes above-the-fold, es decir, aquellas visibles sin necesidad de scroll en la carga inicial, no deben llevar **loading="lazy"**, ya que esto retrasaría su descarga y perjudicaría precisamente las métricas que se pretende mejorar. En este proyecto se aplicó **loading="lazy"** únicamente a las imágenes que requerían scroll para ser vistas, manteniendo sin el atributo las imágenes hero y las primeras tarjetas de cada página.

4.2 Carga asíncrona de scripts

¿Qué sucede al aplicar carga asíncrona a los scripts de tu página? ¿Qué problemas crees que podrían surgir si cargas el JavaScript de forma asíncrona? Ten en cuenta los diferentes métodos de carga asíncrona para responder esta pregunta.

Al aplicar carga asíncrona a los scripts, el comportamiento del navegador cambia fundamentalmente respecto al modo de carga por defecto.

Un **<script src="...">** sin atributos adicionales bloquea el parser HTML en el momento en que el navegador lo encuentra: detiene la construcción del DOM, descarga el script y lo ejecuta antes de continuar. Esto es especialmente problemático cuando el script está en el **<head>**, ya que retrasa el renderizado de todo el contenido visible.

Existen tres aproximaciones para la carga no bloqueante, con comportamientos distintos:

Con **async**, el script se descarga en paralelo al parsing del HTML, pero se ejecuta en cuanto está disponible, interrumpiendo el parser en ese momento. No hay garantía de orden de ejecución entre múltiples scripts con **async**, el primero en descargarse es el primero en ejecutarse. Esto lo hace adecuado para scripts completamente independientes, como analytics o scripts de terceros sin dependencias.

Con **defer**, el script también se descarga en paralelo, pero su ejecución se pospone hasta que el DOM está completamente construido, y los scripts se ejecutan en el orden en que aparecen en el HTML. Esto lo hace seguro para scripts que manipulan el DOM, ya que garantiza que los elementos existen en el momento de la ejecución.

Con **type="module"**, que es la aproximación adoptada en este proyecto, el comportamiento es equivalente a **defer** por especificación: descarga paralela y

ejecución diferida tras la construcción del DOM. Adicionalmente, los módulos ES permiten declarar dependencias explícitas mediante **import**, lo que garantiza el orden de carga y ejecución sin necesidad de gestión manual. Cada módulo solo se descarga si es importado, lo que reduce el código enviado al navegador.

Los problemas que pueden surgir con la carga asíncrona son principalmente dos. Con **async**, si un script depende de otro (por ejemplo, un script que usa una función definida en otro archivo), la ejecución fuera de orden puede provocar errores de referencia. En este proyecto se experimentó este problema en la fase sin optimizaciones: al colocar el script en el **<head>** sin **defer** ni **DOMContentLoaded**, el código intentaba acceder a elementos del DOM que aún no existían (`document.querySelector('.header')` devolvía `null`), lo que interrumpía la ejecución y dejaba la página sin funcionalidad de header ni animaciones AOS. Con **defer** o **type="module"** este problema desaparece porque la ejecución siempre ocurre con el DOM completo.

4.3 Carga asíncrona de css

No hemos hecho carga asíncrona de estilos. ¿Crees que se podría hacer?

¿Qué problemas podríamos tener? Razona tu respuesta.

Técnicamente sí es posible cargar CSS de forma asíncrona. La técnica más utilizada consiste en asignar `media="print"` al elemento `<link>` y cambiar el atributo a `all` una vez descargado mediante el evento `onload`:

```
<link rel="stylesheet" href="styles.css" media="print" onload="this.media='all'">
```

```
<noscript><link rel="stylesheet" href="styles.css"></noscript>
```

Al declarar `media="print"`, el navegador descarga la hoja de estilos sin bloquear el renderizado, ya que la considera no aplicable al medio actual (pantalla). Cuando termina la descarga, el evento `onload` cambia el `media` a `all`, activando los estilos. El elemento `<noscript>` garantiza que los estilos se carguen igualmente si JavaScript está desactivado.

Sin embargo, esta técnica presenta un problema fundamental: el **FOUC** (*Flash of Unstyled Content*). Durante el intervalo entre el inicio del renderizado y la aplicación de los estilos, el usuario ve el HTML sin ningún estilo aplicado — texto sin tipografía, sin colores, sin layout. Esto es especialmente visible en conexiones lentas y resulta muy perjudicial para la experiencia de usuario.

La solución estándar para mitigarlo es el *critical* CSS: extraer manualmente el CSS mínimo necesario para renderizar correctamente el contenido above-the-fold e inlinearlo directamente en el <head> como <style>, cargando el resto de la hoja de estilos de forma asíncrona. De este modo el usuario ve el contenido inicial correctamente estilado mientras el CSS completo se descarga en segundo plano. En este proyecto no se implementó la carga asíncrona de estilos por dos razones. Primera, Parcel genera los bundles CSS automáticamente con hashes en el nombre de archivo, lo que impide referenciarlos directamente sin un proceso de extracción adicional. Segunda, y más importante, el FOUC resultante degradaría significativamente la experiencia de usuario en una web cuyo diseño visual es parte central del proyecto. La relación coste-beneficio no justifica la implementación dado el score de rendimiento ya obtenido.

5. Anexos

1. Aplicación web. Saborea México (Netlify)

<https://saboreamexico.netlify.app/>

2. Repositorio con el código (GitHub):

<https://github.com/marcturu/saboreamexico>

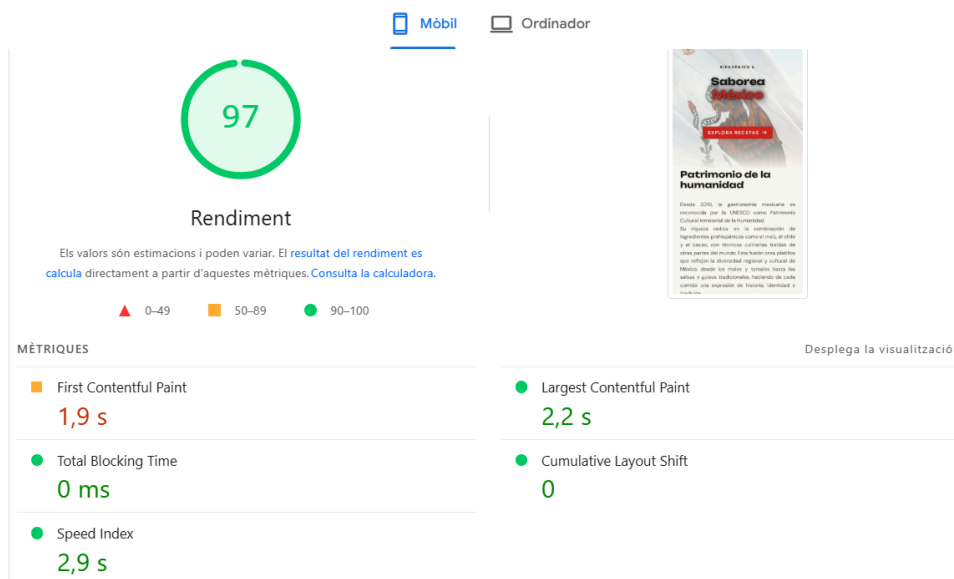
3. Captura de pantalla de la inspección en Firefox Developer de "/" en las fase 1:

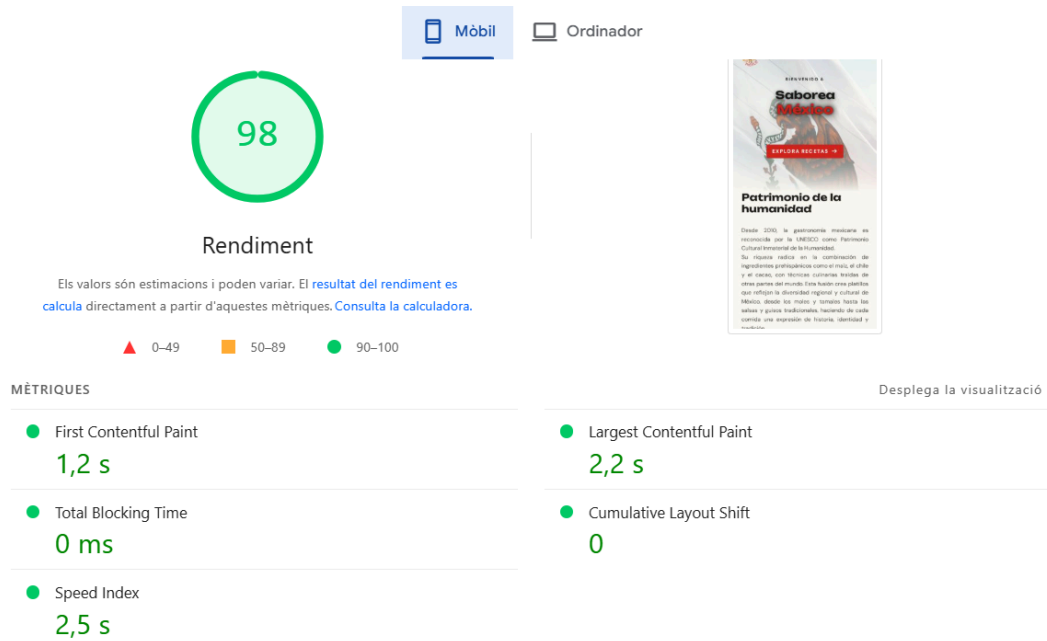
Estado	Método	Dominio	Archivo	Iniciador	Tipo	Transferido	Tamaño	0 ms	5,12 s
200	GET	uoc-ehtmlcss1-mar...	dm-sans-v17-latin-700.69f551ef.woff2	font	woff2	14,35 KB	14,35 ...	3028 ms	
200	GET	uoc-ehtmlcss1-mar...	unbounded-v12-latin-700.e2da09a.woff2	font	woff2	21,09 KB	21,09 ...	3018 ms	
200	GET	uoc-ehtmlcss1-mar...	fa-solid-900.d90600d1.woff2	font	woff2	114,74 KB	114,74...	4881 ms	
200	GET	uoc-ehtmlcss1-mar...	dm-sans-v17-latin-regular.5a445b02.woff2	font	woff2	14,20 KB	14,20 ...	3018 ms	
200	GET	uoc-ehtmlcss1-mar...	android-chrome-512x512.ab6da69e.png	img	png	221,03 KB	221,03...	6360 ms	
200	GET	uoc-ehtmlcss1-mar...	favicon-32x32.ab2e32f4.png	img	png	2,95 KB	2,95 KB	4324 ms	

4. Captura de pantalla de la inspección en Firefox Developer de "/" en las fase 2:

Estado	Método	Dominio	Archivo	Iniciador	Tipo	Transferido	Tamaño	0 ms	2,56 s	5,12 s
200	GET	uoc-ehtmlcss1-mar...	dm-sans-v17-latin-700.69f551ef.woff2	font	woff2	14,35 KB	14,35 ...	1025 ms		
200	GET	uoc-ehtmlcss1-mar...	unbounded-v12-latin-700.e2da09a.woff2	font	woff2	21,09 KB	21,09 ...	1025 ms		
200	GET	uoc-ehtmlcss1-mar...	fa-solid-900.d90600d1.woff2	font	woff2	114,74 KB	114,74...	2049 ms		
200	GET	uoc-ehtmlcss1-mar...	dm-sans-v17-latin-regular.5a445b02.woff2	font	woff2	14,20 KB	14,20 ...	3019 ms		
200	GET	uoc-ehtmlcss1-mar...	android-chrome-512x512.ab6da69e.png	img	png	221,03 KB	221,03...	4373 ms		
200	GET	uoc-ehtmlcss1-mar...	favicon-32x32.ab2e32f4.png	img	png	2,95 KB	2,95 KB	4375 ms		

5. Capturas de pantalla de tests de Google Insights de "/" en móvil en fase 2 y fase 3, respectivamente:





6. Captura de pantalla de tests de Google Insights de “/det2” en móvil en fase 3:

